

1. DBMS → A software that provides services like creating and managing data.
2. Database System → DBMS software with database.
3. Traditional → Only Numerical and textual databases. Nowadays data became more than numerics and text.
4. File based system and database system both uses files to store data. The way they handle data makes them different.
5. File Based System
 - a. **Redundant data will be there.** Mone kor ekta school er exam dept student grades rakhtese file e and Finance dept student fee rakhtese arek file e. Both are using students name, id, phone number but as they both are in separate file, they can not access each others data. Because of that we will need more space as they are writing same file.
 - b. **Cant update all file in a single click.** Ekhn mone kor student bollo tar name bhul ashce grade sheet e. So exam dept ekta click kore shob jaigai name update kore dite parbe na cause everyone has different files, so oi student ke shob dept e giye personally update korte hobe. Which is time consuming.
 - c. Basically the data will not be up to data, need redundant efforts to update data and more space for a common data.
 - d. Here use has to write all the procedures for managing the database. And because of that the user will be able to see everything in detail.
 - e. No crash mechanism. If something happens data will be gone,
 - f. No permisson mechanism. Like you can alter permission and give access to anyone.
6. Database system
 - a. A common database will be there and everyone will be able to access the data and use it as they need.
 - b. Here it will save time and space. And it will be easy to update.
 - c. User will not require to write any type of code. Administrator will write it for you. Because of that user will not see everything.
 - d. Has a crash recovery machanism. If system fails data can be recoverd.
 - e. Has a permission mechanism. Using this we can control how a certain user will be able to access the file.
7. Features of DBMS
 - a. Reduce redundancy
 - b. Define a particular database in terms of data types, structures and constrains.
 - c. Can manipulate the data - retrieval. Modification,
 - d. Can process and share data to many users, yet keeping all data valid and consistent.
 - e. Can provide presentation and visualization of data in multiple ways
 - f. Provides protection of data

8. Meta data → Data about data. It returns the information about what types of data is in DBMS. this helps DBMS software to work with different DBMS application. The DBMS stores the descriptions of the schema constructs and constraints also called the meta-data.
9. Data Abstraction → Data abstraction generally refers to the **suppression of details of data organization and storage, and the highlighting of the essential features** for an improved understanding of data.
10. OLTP → online transaction process, it allows hundreds of concurrent transactions to execute per second.
11. Concurrency control → it guarantees that each transaction is correctly executed or not.
12. Main characteristics of Database approach →
 - a. Self describing nature → DBMS has metadata
 - b. Insulation between program and data → program data independence
 - c. Data abstraction → will only show the data that users need.
 - d. Support of multiple views of the data → admin can control which user see what
 - e. Sharing data and multi-user transaction processing → multiple user can retrieve and update simultaneously. Also it can process multiple transaction at a time.
13. Advantage of using database →
 - a. Sharing and update data among multiple users. Which will reduce redundancy.
 - b. Controlling what a user will see. Multiple interfaces for different user class.
 - c. Providing storage structure for efficient queries.
 - d. Backup and recovery service.
 - e. Representing complex relationships among data.
14. Database user
 - a. Actors on the scen → who actually use DBMS
 - i. Database administrators → Those who wil control and monitor everything of a database like where should data get stored or which software to use.
 - ii. Database Designer → those who will define the content, structure and build the blueprint of a DBMS. They are responsible to communicate with end user and develop what

- iii. Software Engineers → Responsible for implementing the database and actually build it using programming language and tools.
- iv. End user → Those who will use the content of database.
 - 1. Casual → access database when needed. Ex - middle or higher level managers
 - 2. Naive → Do not know how DBMS works but they use everyday in their life. Most of the end users are here. Ex - facebook user
 - 3. Sophisticated → familiar with database and has capability to build their own database. They do not write program code but do use SQL queries directly.
 - 4. Stand alone → they make their own database application because their data do not fit on traditional format.
- b. Workers behind scene → who designs and develops the DBMS software and tools

- Main inhibitors (costs) of using a DBMS:
 - High initial investment and possible need for additional hardware
 - Overhead for providing generality, security, concurrency control, recovery, and integrity functions
- When a DBMS may be unnecessary:
 - If the database and applications are simple, well defined, and not expected to change
 - If access to data by multiple users is not required
- When a DBMS may be infeasible
 - In embedded systems where a general-purpose DBMS may not fit in available storage

- When no DBMS may suffice:
 - If there are stringent real-time requirements that may not be met because of DBMS overhead (e.g., telephone switching systems)
 - If the database system is not able to handle the complexity of data because of modeling limitations (e.g., in complex genome and protein databases)
 - If the database users need special operations not supported by the DBMS (e.g., GIS and location-based services).

16.

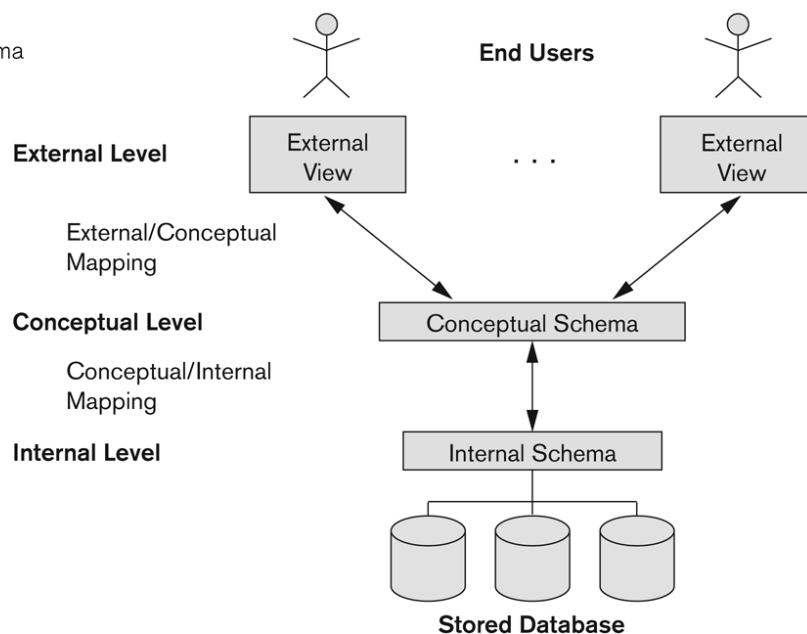
17. Data Model → Data models in DBMS help to understand the design at the conceptual, physical, and logical levels as it provides a clear picture of the data making. So it becomes easier for developers to create a physical database and also non technical people to understand if their requirements getting fulfilled or not. Data models are used to describe how the data is going to be stored, accessed, and updated in a DBMS. basically **A set of concepts to describe the structure of a database.**

18. Categories of data models →

- a. Conceptual → high level, provides the design of database without any technical and details term. **It provides overview of concept of the database.**
- b. Representational/ Logical → **provides the description** of tables, object oriented classes, xml tags, document structure and many more. document structures are defined in this model. **Representational data models hide many details of data storage on disk but can be implemented on a computer system directly.**
- c. Physical → low level, **provides the actual detail about physical structure to store the data.** Like partitions, CPU spaces etc
- d. Self describing data models → **The data storage in systems based on these models combines the description of the data with the data values themselves.**

19. Database schema → **description of database** which includes database structure, data types and constraints on the database. It changes very frequently and also known as intension.
20. Schema Diagram → illustrative display of database schema.
21. Database state/occurrence/instance → **The data in the database at a particular moment in time** is called a database state or snapshot. It shows actual data stored in a particular moment. It is also known as database instance. It changes whenever database is updated.
22. Three schema architecture → it does not explicitly used in commercial DBMS products. it is made for two reasons
- Program data independence
 - Support multiple views of the data
23. DBMS schema three level
- External schema → uses conceptual data model.
 - Conceptual schema → describe structure and constrains for the whole database for a community of user. Uses implementation data model
 - Internal schema → describes physical storage structures and access path. Uses physical data model.

Figure 2.2
The three-schema architecture.



- d.
24. Data independence → Data independence can be achieved if you can change lower level scheme without affecting higher level scheme. For example, in three level scheme if i change anything in physical level and if it does not affect logical level then physical data

independence achieved. On the other hand, if we change logical data and if it does not affect external schema then logical data achieved. Data is separated from the programs, so that the changes made to the data will not affect the program execution and the application.

- a. Logical data independence → capacity to change conceptual schema without having to change external schema. Ex- adding new attribute without changing existing application, merging two records into one, or dividing two records, Add/Modify or Delete a new attribute.
- b. Physical data independence → capacity to change internal schema without having to change conceptual schema. Ex - changing location of database, modifying index, switching to new ds, using new storage. Change in compression techniques, Hashing algorithms and storage devices etc.

25. DDL VS DML → <https://www.geeksforgeeks.org/difference-between-ddl-and-dml-in-dbms/>

DBMS Interfaces

- Stand-alone query language interfaces
 - **Example: Entering SQL queries at the DBMS interactive SQL interface (e.g. SQL*Plus in ORACLE)**
- Programmer interfaces for embedding DML in programming languages
- User-friendly interfaces
 - **Menu-based, forms-based, graphics-based, natural language interfaces, etc.**
- Mobile Interfaces: interfaces allowing users to perform transactions using mobile apps
- Parametric interfaces, e.g., bank tellers using function keys.
- Interfaces for the DBA.

26.

27. Difference between database schema and database state

The distinction between database schema and database state is very important. When we **define** a new database, we specify its database schema only to the DBMS. At this point, the corresponding database state is the *empty state* with no data. We get the *initial state* of the database when the database is first **populated** or **loaded** with the initial data. From then on, every time an update operation is applied to the database, we get another database state. At any point in time, the database has a *current state*.⁸ The DBMS is partly responsible for ensuring that every state of the database is a **valid state**—that is, a state that satisfies the structure and constraints specified in the schema. Hence, specifying a correct schema to the DBMS is extremely important and the schema must be designed with utmost care. The DBMS stores the descriptions of the schema

28.

29. Question →

- a. Difference between data level structure and three schema architecture?

Lecture -5

1. **Key Constraint** : Values should be Unique. Can take NULL values unless it's a Primary Key and NOT NULL constraint has not been set.
2. **Entity Integrity Constraint** : Values should be Unique and NOT NULL.
 - **NOT NULL** - Ensures that a column cannot have a NULL value
 - **UNIQUE** - Ensures that all values in a column are different
 - **PRIMARY KEY** - A combination of a **NOT NULL** and **UNIQUE**. Uniquely identifies each row in a table
 - **FOREIGN KEY** - Prevents actions that would destroy links between tables
 - **CHECK** - Ensures that the values in a column satisfies a specific condition
 - **DEFAULT** - Sets a default value for a column if no value is specified
 - **CREATE INDEX** - Used to create and retrieve data from the database very quickly
- 3.
4. Unique e NULL ashte pare. And multiple NULL ashte pare